

The final security problem in IOT : Don't count on the canary!

1st Qian Zhou

1, Key Laboratory of Safety-Critical Software (NUAA)
Ministry of Industry and Information Technology
2, Nanjing University of Posts and Communications
Nanjing, China
zhouqian@njupt.edu.cn

2nd Hua Dai

School of Computer Science
Nanjing University of Posts and Communications
Nanjing, China
daihua@njupt.edu.cn

3rd Liang Liu

College of Computer Science and Technology
Nanjing University of Aeronautics and Astronautics
Nanjing, China
liangliu@nuaa.edu.cn

4th Kai Shi

1, China Electronics Technology Cyber Security Co.,Ltd
2,30 Institute of CETC
Chengdu, China
1250535163@qq.com

5th Jie Chen

1, 30 Institute of CETC
2, NorthWestern Polytechnical University
Chengdu, China
lucycc777@21cn.com

6th Hong Jiang

1, China Electronics Technology Cyber Security Co.,Ltd
2, 30 Institute of CETC
Chengdu, China
1213@163.com

Abstract—Memory-based vulnerabilities are becoming more and more common in low-power and low-cost devices in IOT. We study several low-level vulnerabilities that lead to memory corruption in C and C++ programs, and how to use stack corruption and format string attack to exploit these vulnerabilities. Automatic methods for resisting memory attacks, such as stack canary and address space layout randomization ASLR, are studied. These methods do not need to change the source program. However, a return-oriented programming (ROP) technology can bypass them. Control flow integrity (CFI) can resist the destruction of ROP technology. In fact, the security design is holistic. Finally, we summarize the rules of security coding in embedded devices, and propose two novel methods of software anomaly detection process for IOT devices in the future.

Keywords—Buffer overflow, ROP, CFI, Buffer overflow, Anomaly detection

I. INTRODUCTION

In the era of interconnection of all things, more and more intelligent devices appear in cyberspace. Technology has brought convenience to our life. For example, Internet of things devices such as smart appliances or vehicles will work in the way we expect. The reliability of the behavior of these agents is an important goal of the system when the application runs. With the enhancement of system and network inter-operability, advanced persistence threatens the existence of apt, and reliable behavior has evolved into a security problem of space ecosystem[1]–[2]. Each behavior should run as expected, and every dimension to be applied from the bottom may be maliciously exploited. Over the years, with the implementation of various protection mechanisms such as host based and network-based IDS and IPS, mature operating systems have stronger ability to detect and infer attacks and penetration, and

limited Internet of things devices have become a thorny weak link.

Memory attack vulnerabilities are becoming more and more common in low-power and low-cost Internet of things devices[3]–[5]. In the CWE ranking in 2021, cwe-787 and out of Bounds Write topped the list, and cwe-125out of Bounds Read ranked among the top three. Due to fine-grained memory management, low running overhead and strong portability, C and C++ are the general programming languages of most IOT device micro-controllers. At the same time, they are also many embedded operating system development languages, such as industrial control system, automobile, space probe and so on. Research shows that nearly 80% of the vulnerabilities exist in software applications, and the launched programs allocate their own program segments in memory. However, when a piece of data in its storage area is modified, the behavior of the program no longer meets the expectation. This effect is called "memory corruption", and hackers who trigger this change can use it for their own benefit.

We study several low-level vulnerabilities that lead to memory corruption in C and C++ programs, and how to use stack corruption and format string attack to exploit these vulnerabilities; (2) Automatic methods to resist memory attacks, such as stack canary and address space layout randomization ASLR, are studied. These methods do not need to change the source program. However, a return-oriented programming (ROP) technology can bypass them. 3) the study of control flow integrity (CFI) can resist the destruction of ROP technology. In fact, the security design is holistic. Finally, 4) summarizes the rules of security coding in Internet of things devices, and propose two methods of software anomaly

detection process for the security program in Internet of things devices in the future.

II. MEMORY EXPLOIT IN IOT

If the input and execution are memory safe, the program is memory safe. Memory security is a feature of program execution. Program execution is memory safe. There are two prerequisites for security attributes: the first is space security attribute, which means that the pointer can only access the memory owned by the pointer. The second condition is roughly what we call the attribute of time security. It must construct a valid pointer, which is a pointer returned by the standard heap *allocator* and *malloc*, or a

pointer created by legal language operations (such as *and* or *operator* addresses) and dot arithmetic.

A. Spatial Safe: Buffer overflow

A buffer is simply a contiguous area of memory associated with a program variable or field. The layout of C program in memory is shown in the Fig.1. When the program tries to write more data into the buffer it can actually save, overflow will occur. Buffer overflow is related to safety. What happens when a program reads or writes to a buffer beyond its boundaries? According to the C programming language standard, such a program is undefined. In fact, it can do anything. So basically, we can cover a lot of memory on the stack. The question is, if you are an attacker, what can you

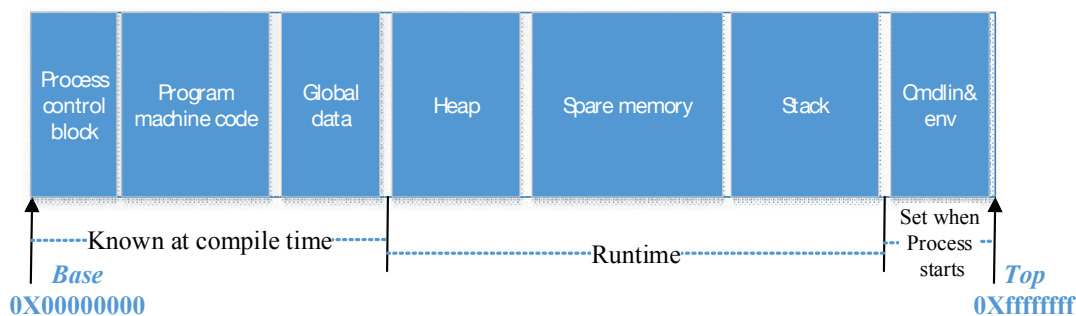


Fig. 1. Memory layout

can do is overwrite the buffer with code, and an attacker can take advantage of such memory vulnerabilities to control the whole system. The following describes the typical memory attacks in the firmware of Internet of things devices in recent years.

Stack Buffer overflow

Stack buffer overflow is mainly caused by stack smashing attack. The buffer overflow vulnerability analysis method based on finite state machine (FSM) can statically analyze the source code. For two types of buffer overflow vulnerabilities caused by function call error and circular copy error, the corresponding vulnerability analysis model is designed[6]. The most direct mitigation measure of memory damage vulnerability is repair. Vulnerability prevention technology: static analysis, compiler modification, operating system modification and hardware modification, and detect and clear the source code before using the software; Source code audit[3]. Software diversity (ASD) is realized by randomizing the program (i.e., randomizing the program data space or control flow sequence), and preventing memory leakage caused by code reuse attack by adding probability protection to binary files.

Heap Buffer overflow

Unlike stack overflow, heap overflow has no return address[7]. For memory allocation and recycling, it is difficult to detect. The basic idea of heap defender is to extract the predefined heap memory range, and then check whether the heap operation address of the instruction exceeds the heap

address range when the embedded processor is running[8]. Prober method focuses on heap objects related to arrays, marks all vulnerable call stacks during compilation, and then uses page protection to detect any invalid access during run-time[9].

Global Buffer overflow

There are two places where global and static variables can be located: segment for initialized data, and segment for uninitialized data (BSS segment). Because of the lightweight nature of Internet of things applications, attacks on global and static variables are relatively limited. According to the idea of separating data in the data segment according to the data type, [10] propose the idea of separation areas are isolated through a protection page. This prevents buffer overflows from overwriting data or code pointers, thereby preventing attackers from performing code injection attacks. However, the performance and memory overhead do not meet the real-time requirements of the Internet of things.

Generally speaking, for spatial security, the Data Execution Prevention (DEP) mechanism can be used to specify the execution permission of the memory location. This protection prevents malicious code from executing directly from the buffer value by allowing only specific memory locations to have execution permission. Address space layout randomization (ASLR) is implemented by binary files or operating systems. This protection

mechanism randomizes the memory addresses of binary and external libraries at each execution.

B. Temporal safe: Dangling pointer

A time security violation occurs when a program accesses undefined memory. Undefined memory refers to memory that has never been allocated or previously allocated but has now been released. Stack buffer overflow in most cases will overwrite the return address of the function call on the stack. Heap overflow is very similar to stack overflow, but there is no return address. Common memory problems of dangling pointers include double free, invalid free, stack use after free, heap use after free, etc. Method [11] is implemented by tracking the pointers of each memory object and replacing them with invalid pointers after the objects are released. In addition, it is also possible to change the pointer to its alias by using the middle pointer to zero, which can alleviate the vulnerability caused by dangling the pointer [12].

C. Type safety

Type safety is usually guaranteed by mechanisms such as garbage collection, boundary checking and abstract representation. These mechanisms make programming easier and programs easier to reason, but they increase memory and time overhead. In other words, a new generation of languages is emerging, which not only provides type safety, but also aims at good performance. For example, Mozilla's rust programming language allows programmers to avoid garbage collection in some cases.

Format string overflows

When the process is running, the read-write cross-border defense of the formatted string during the execution of the system, and the monitoring from the program memory layout, processor execution, system call and other aspects of the run-time system are the concerns of the read-write cross-border dynamic defense technology of the formatted string. So far, the detection technologies of format string vulnerabilities mainly include the following categories: 1) Lexical analysis technology; 2) C library enhancement technology; 3) Restricted write technology; 4) Attack immunity technology; 5) Type qualifier technology[13].

Integer overflows

Integer overflow is an arithmetic overflow error that occurs when the result of integer operation does not match the allocated memory space. It usually leads to unexpected results rather than errors in the program. Only the signed number will overflow, and the unsigned number will wrap around, often bypassing some judgment statements. After integer wrapping, the index will be out of bounds and uncertain data will be obtained. This vulnerability is because integers are stored in a fixed length space in memory, and the maximum and minimum values it can store are fixed. If we try to store a number, and the number is greater than the fixed maximum value, it will lead to integer overflow. Method

[14] allows the sanitized code to filter out integer overflows before performing integer overflow tests. [15] uses inter program data flow analysis and pollution analysis to accurately identify potential integer overflow vulnerabilities.

D. Return Oriented Programming, ROP

Shacham proposed the idea of return-oriented programming (ROP), which is based on the principle of control flow hijacking. By using the effective short instruction sequence ending with RET, JMP and other instructions in the program library, shacham constructed the gadget set to complete the calculation and attack. ROP will bypass the separation of address space. ROP attack detection methods include static and dynamic[16]: analyze the program code through lexical analysis, syntactic analysis, control flow and data flow analysis without running the code. We can get the instruction address, control flow diagram, block, jump node, section, etc. of a program. Malformed input is used to hijack program control flow[4]. Dynamic method short instruction sequence usage frequency check, call integrity check[19, 20], etc. The ROP defense technology. Based on the arm platform arranges the absolute address of gadgets in memory to the stack, and jumps to this address to execute the code when intercepting the program flow, so that the whole ROP chain fails and the attack fails.

III. SYSTEM ARCHITECTURE

Memory attacks often rely on the underlying architecture. At present, the memory architecture of IOT device processor is mainly divided into von Neumann and Harvard model. DSP devices are modified Harvard architecture, such as X-ray, fitness tracks, smart watches and digital assistants, such as Xiaomi speakers, and Google home, what these DSP applications have in common is the capture and digitization of analysis information followed by some form of processing of the digital signal generated.

A. Architecture

Von Neumann architecture. The von Neumann architecture is based on the computer architecture described by John von Neumann and others in the first draft of the EDVAC report in 1945. The memory where data and instructions are stored together. Instruction acquisition and data operation cannot occur at the same time (because they share a common bus). This is known as the von Neumann bottleneck and usually limits the performance of the corresponding system.

Harvard model architecture. Harvard architecture is also a stored program system, but different from von Neumann architecture, it has a set of dedicated address and data buses for reading and writing to memory, and a set of address and data buses for obtaining instructions[18]. Harvard structure can be used to complete dynamic programs and complex system requirements. Since ARM9, Harvard architecture has been designed inside the processor, which solves the conflict between simultaneous finger and data retrieval in DSP and other processors that often use convolution operation. While

improving the parallel efficiency, it protects the security of instruction execution.

Modified Harvard Architecture. Many modern embedded devices have modified Harvard architecture, which is different from Harvard architecture[19]. These modifications are various ways to relax the strict separation between code and data, and still support higher performance concurrent data and instruction access in Harvard architecture. For example, the most common modification is to build a memory hierarchy that provides separate CPU caches for instructions and data at lower levels of the hierarchy. There is only one address space for instructions and data, which provides von Neumann model, but the CPU obtains instructions from the instruction cache and obtains data from the data cache. Another obvious difference between the improved Harvard architecture and the pure von Neumann architecture is that when instructions are executed from one memory segment, the same memory segment cannot be accessed as data at the same time. This advantage makes the execution of the program unaffected after the data field is overwritten, so it can resist ROP attacks

B. Operating system

The operating system of IOT is to manage all the hardware and software on the embedded device and provide all the resources it needs, such as processing, memory and storage[20]. The design of embedded operating system also determines the risk of memory leakage. Representative operating systems include Contiki, FreeRTOS and mbed OS. Contiki is very useful in building complex wireless systems, and memory efficiency is also high. FreeRTOS uses Amazon Web services (AWS IOT core) to run IOT applications. Its memory footprint is particularly small (only 6-15kb), making it a more adaptable small microcontroller. Mbed OS is a free open-source operating system, which is widely recognized because it uses arm cortex m and provides a variety of connection options that developers can use, including WiFi and Bluetooth. Its multi-layer security protocol is very attractive to developers. Modern operating systems prevent buffer overflow. The operating system will also prohibit the execution of data space - where it is embedded in the program stack. Stack canary and address space randomization are such protection. A stack canary is a random number placed on the stack during a call. Returning ensures that the stack Canary remains unchanged. Overwriting the return address requires overwriting the stack canary.

IV. DEFENSE AND MITIGATION

It is not enough to rely on manual or static methods to attack the memory exploitation of Internet of things devices. People are looking for intelligent active defense methods. Compared with dynamic detection such as instrumentation, active defense does not need to change the original code, but we can't completely rely on them to prevent

vulnerabilities from being exploited. Therefore, as a supplement to automation measures, developers should first try to avoid introducing vulnerabilities into their programs. Next, we first trace the principle of secure coding, and then use static and dynamic methods to detect software behavior anomalies. These three aspects are described below.

A. Security coding rules

The first step to protect your software against buffer overflow attacks is to write stable and robust code that is secure. There are four basic rules needed to be checked.

1) *Enforce input Compliance*: A Fuzzer generally tries a combination of various special characters, alphabets, numerals, pure binary, metadata to check if an application is vulnerable.

2) *Robust coding*: Avoid depending anything around and if something happened, the system will not crash.

3) *Use safe string function*: Printf, sprintf, strcat, strcpy, and gets are examples of functions responsible for these attacks. To altogether avoid overrun attacks on their production application, developers should avoid using these unsafe functions and use their safer versions, i.e., fgets, sprintf_s, strcpy_s, and strcat_s.

4) *Don't forget NUL terminator*: Correctly understand pointer algorithm. Ensure that memory is properly managed, i.e. there are no memory leaks

B. Abnormal behavior detection of dynamic and static combination

The verification method based on control flow or short sequence is still the mainstream method of program dynamic analysis. When the software program is actually executed, the ordinary function and system function call sequence are obtained, and the function call sequence is divided into the mixed call sequence of ordinary function and system function in the unit of function according to the pile insertion, which is matched with the automata corresponding to the function. If it can be accepted by the automata, the software is considered to be running normally. If it cannot be accepted by the automata, it is determined that the software is attacked or abnormal.

Stage 1: normal mode construction stage of function call sequence:

a) *Step A*, Static analysis generates the control flow diagram of ordinary function calls: Based on LLVM compiler, the control flow diagram of each ordinary function call can be directly generated.

b) *Step B*, Static analysis generates the system call control flow diagram of ordinary functions: the control flow diagram of C or C++ code can be generated based on LLVM. First compile the program to LLVM IR, and then generate the control flow diagram using opt. The control flow diagram is a file in dot format. We read the dot file generated by it and read each element in the graph in turn, judge that the element is node information or node pointing information, and build a directed graph. Then we will traverse the nodes in turn to judge whether the node contains system calls. If not, add all child nodes of the node

to the child node set of its parent node, and delete the node. Finally, the newly constructed directed graph is used to regenerate the control flow graph containing only system calls.

c) *Step C*, Merge to obtain the mixed control flow diagram: Based on the control flow diagram of ordinary function call generated in step B, insert the system call control flow diagram of each ordinary function generated in step C into the control flow diagram of ordinary function call. Then a complete control flow diagram of mixed calls of ordinary functions and kernel functions is obtained.

d) *Step D*, The control flow graph of mixed calls is transformed into automata: when the node has only one system call, it is transformed to identify this system call from the current state to the next state. When the node contains multiple system calls, the state is added, and $n-1$ states are added for n system calls to expand, which meets the definition of DFA. Each function corresponds to an automata, so we get the automata of all functions.

Stage 2: program abnormality determination

e) *Step E*, Program stake insertion: stake insertion is carried out at the beginning and end of the entry function of each function of the program and the direct or indirect call function of the entry function, so as to facilitate the subsequent division of the system function call sequence into short sequences.

f) *Step F*, Obtain the system function call sequence during program dynamic execution: use p-trace to obtain the system function call sequence during program execution, and divide the whole system function call sequence into short sequences corresponding to each function according to the pile insertion information.

g) *Step G*, Judging whether the program is abnormal or not: according to the short sequence divided in Step F each short sequence is corresponding to the automata of each function obtained in step D. if each short sequence can be recognized by the corresponding automata, it is considered that the software is running normally. If it cannot be accepted by the automata, it is judged that the software is under attack or abnormal execution. At this time, the software behavior exception warning is triggered and the active protection mode is entered.

C. Dynamic abnormal behavior detection based on Multi-threadings

With the intellectualization of IOT devices, many programs are executed concurrently. Single path detection often has limitations. Due to the influence of thread scheduling, when a program runs multiple times under the same input, the system call sequence may be different, which cannot accurately represent the behavior of the program. For multithreaded programs, the influence of thread scheduling on system call sequence is shielded, and the running status of multithreaded programs is detected in real time by thread level system call sequence. This paper also proposes a multi-threaded program

exception detection method based on system call sequence. The main steps are as follows:

1) S1: Code Instrumentation

In order to obtain the system call information when the program is running, it is necessary to use the pile insertion tool to insert the pile into the program. Program plug-in refers to inserting code segments into the program on the basis of ensuring the original logic of the program and not changing the original function. These code segments can detect the information when the program is running, such as inserting detection code at the entrance of system call. When the program calls system call, the detection code can obtain the information of caller and system call, including thread number of thread, system call name, system call number and so on. Thus, the system call sequence of each thread that was originally randomly mixed can be extracted by taking the thread number as the index.

2) S2: Get trusted thread level system call sequence

In order to learn the normal behavior of the program, the program needs to be run in a safe and trusted environment. The so-called safe and trusted environment mainly means that the program will not be attacked by the outside or can run normally even if attacked. Building a safe and trusted environment generally requires the support of hardware. In a safe and trusted environment, run the program after pile insertion for many times, traverse all or most of the execution paths of the program as much as possible, and obtain the system call sequence containing thread information generated during program operation; According to the thread number in the thread information, the total system call sequence is disassembled to obtain the system call sequence generated by each thread.

3) S3 : Mining behaviors in thread level system call sequences

Var-grams algorithm is used to segment the thread level system call sequence obtained in step S2. Each thread level system call sequence will be divided into several short system call sequences with different lengths. Each short system call sequence represents a behavior of the program; Put all the short sequences of system calls into the set to build a dictionary set of program behavior. Var grams algorithm segments the system call sequence and extracts the form of patterns. Each pattern is a short sequence after segmentation. A pattern can represent a behavior of the program.

4) S4: Training hidden Markov (HMM) model

The behavior dictionary obtained in step S3 is used as the training data of HMM model for supervised training to obtain various parameters of HMM model; Build the behavior Library of the program.

5) S5: Obtain the behavior when the program is running

Similar to step S2, collect the thread level system call sequence generated by the program running in the real environment; Take the collected thread level system call sequence as the input of HMM model, segment the sequence, and divide the sequence into several short system call sequences with different lengths. Each short sequence

represents a behavior of the program, so as to obtain the behavior sequence of the actual operation of the program.

6) S6: Anomaly Detection

Match the detected behavior when the program is actually running with the behavior in the behavior library, and judge whether the program is abnormal according to the matching results.

V. CONCLUSION

We study how to use stack corruption and format string attack to exploit these vulnerabilities. Automatic methods to resist memory attacks, such as stack canary and address space layout randomization ASLR, are studied. However, a return-oriented programming (ROP) technology can bypass them. CFI can resist the destruction of ROP technology. None of these defense measures is perfect. Even CFI cannot defeat the attack on memory security. In fact, the security design is holistic. Finally, we summarize the rules of security coding in Internet of things devices, and propose two methods of software anomaly detection process for the security program in Internet of things devices in the future.

ACKNOWLEDGMENT

The authors thank the support by Nanjing University of Posts and Telecommunications. The work is also sponsored by the National Natural Science Foundation of China (Grant Nos. 61902199 and 61872197); and Key Laboratory of Safety-Critical Software (Nanjing University of Aeronautics and Astronautics) Ministry of Industry and Information Technology.

REFERENCES

- [1] Y. Al-Hadhami and F. K. Hussain, "DDoS attacks in IoT networks: a comprehensive systematic literature review," *World Wide Web*, vol. 24, no. 3, pp. 971–1001, May 2021, doi: 10.1007/s11280-020-00855-2.
- [2] H. Zhang, W. Zhang, Z. Lv, A. K. Sangaiah, T. Huang, and N. Chilamkurti, "MALDC: a depth detection method for malware based on behavior chains," *World Wide Web*, vol. 23, no. 2, pp. 991–1010, Mar. 2020, doi: 10.1007/s11280-019-00675-z.
- [3] K. V. English, I. Obaidat, and M. Sridhar, "Exploiting Memory Corruption Vulnerabilities in Connman for IoT Devices," in 2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), Portland, OR, USA, Jun. 2019, pp. 247–255. doi: 10.1109/DSN.2019.00036.
- [4] A. Biswas, Z. Li, and A. Tyagi, "Control Flow Integrity in IoT Devices with Performance Counters and DWT," in 2020 IEEE International Symposium on Smart Electronic Systems (iSES) (Formerly iNiS), 2020, pp. 171–176. doi: 10.1109/iSES50453.2020.00046.
- [5] J. Habibi, A. Panicker, A. Gupta, and E. Bertino, "DisARM: Mitigating Buffer Overflow Attacks on Embedded Devices," in *Network and System Security*, Cham, 2015, pp. 112–129. doi: 10.1007/978-3-319-25645-0_8.
- [6] L. Jie, H. Da, and R. Zhihong, "An Analysis Model of Buffer Overflow Vulnerability Based on FSM," in *Proceedings of the 2019 2nd International Conference on Geoinformatics and Data Analysis*, New York, NY, USA, Mar. 2019, pp. 47–51. doi: 10.1145/3318236.3318261.
- [7] D. Tian, X. Li, M. Chen, and C. Hu, "iCruiser: An Improved Approach for Concurrent Heap Buffer Overflow Monitoring," *IEICE Transactions on Information and Systems*, vol. E97.D, no. 3, pp. 601–605, 2014, doi: 10.1587/transinf.E97.D.601.
- [8] D. Li, Z. Liu, and Y. Zhao, "HeapDefender: A Mechanism of Defending Embedded Systems against Heap Overflow via Hardware," in 2012 9th International Conference on Ubiquitous Intelligence and Computing and 9th International Conference on Autonomic and Trusted Computing, Sep. 2012, pp. 851–856. doi: 10.1109/UIC-ATC.2012.115.
- [9] H. Liu, R. Tian, B. Ren, and T. Liu, "Prober: practically defending overflows with page protection," in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, New York, NY, USA, Dec. 2020, pp. 1116–1128. doi: 10.1145/3324884.3416533.
- [10] Y. Younan, F. Piessens, and W. Joosen, "Protecting Global and Static Variables from Buffer Overflow Attacks," in 2009 International Conference on Availability, Reliability and Security, Mar. 2009, pp. 798–803. doi: 10.1109/ARES.2009.126.
- [11] E. van der Kouwe, V. Nigade, and C. Giuffrida, "DangSan: Scalable Use-after-free Detection," in *Proceedings of the Twelfth European Conference on Computer Systems*, New York, NY, USA, Apr. 2017, pp. 405–419. doi: 10.1145/3064176.3064211.
- [12] Y. Wang et al., "DangDone: Eliminating Dangling Pointers via Intermediate Pointers," in *Proceedings of the Tenth Asia-Pacific Symposium on Internetworking*, New York, NY, USA, Sep. 2018, pp. 1–10. doi: 10.1145/3275219.3275231.
- [13] K.-S. Lhee and S. J. Chapin, "Buffer overflow and format string overflow vulnerabilities," *Software: Practice and Experience*, vol. 33, no. 5, pp. 423–460, 2003, doi: 10.1002/spe.515.
- [14] Z. Huang and X. Yu, "Integer Overflow Detection with Delayed Runtime Test," in *The 16th International Conference on Availability, Reliability and Security*, New York, NY, USA, Aug. 2021, pp. 1–6. doi: 10.1145/3465481.3465771.
- [15] M. Xu et al., "A Light-Weight and Accurate Method of Static Integer-Overflow-to-Buffer-Overflow Vulnerability Detection," in *Information Security and Cryptology*, Cham, 2019, pp. 404–423. doi: 10.1007/978-3-030-14234-6_22.
- [16] L. Si, J. Yu, L. Luo, J. Ma, Q. Wu, and S. Li, "ROP-Hunt: Detecting Return-Oriented Programming Attacks in Applications," in *Security, Privacy, and Anonymity in Computation, Communication, and Storage*, Cham, 2016, pp. 131–144. doi: 10.1007/978-3-319-49148-6_12.
- [17] S. Xu, P. Xie, and Y. Wang, "AT-ROP: Using static analysis and binary patch technology to defend against ROP attacks based on return instruction," in 2020 International Symposium on Theoretical Aspects of Software Engineering (TASE), 2020, pp. 209–216. doi: 10.1109/TASE49443.2020.00036.
- [18] Ștefan Nicula and R. D. Zota, "Exploiting stack-based buffer overflow using modern day techniques," *Procedia Computer Science*, vol. 160, pp. 9–14, Jan. 2019, doi: 10.1016/j.procs.2019.09.437.
- [19] N. G. Tsoutsos and M. Maniatakis, "Anatomy of Memory Corruption Attacks and Mitigations in Embedded Systems," *IEEE Embedded Systems Letters*, vol. 10, no. 3, pp. 95–98, Sep. 2018, doi: 10.1109/LES.2018.2829777.
- [20] G. Mullen and L. Meany, "Assessment of Buffer Overflow Based Attacks On an IoT Operating System," in 2019 Global IoT Summit (GIOTS), Jun. 2019, pp. 1–6. doi: 10.1109/GIOTS.2019.8766434.